



Sistema de detecção de intrusão baseado em XAI para detecção de ataques de DNS over HTTPS

Daniel Nascimento da Silva

Centro de Informática (CIn)

Universidade Federal de Pernambuco (UFPE)

Recife, Brasil

dns3@cin.ufpe.br

Lucas Ferreira Alves

Centro de Informática (CIn)

Universidade Federal de Pernambuco (UFPE)

Recife, Brasil

lfa4@cin.ufpe.br

Maria Carolina Santos Berrafato

Centro de Informática (CIn)

Universidade Federal de Pernambuco (UFPE)

Recife, Brasil

mcsb3@cin.ufpe.br

Millena Ferreira Marçal das Neves

Centro de Informática (CIn)

Universidade Federal de Pernambuco (UFPE)

Recife, Brasil

mfmn@cin.ufpe.br

Este trabalho replica e analisa criticamente um sistema de detecção de intrusão para tráfego DNS over HTTPS (DoH) baseado em Machine Learning. O estudo valida a proposta original e estabelece uma base para futuras otimizações voltadas à redução de dimensionalidade e maior capacidade de generalização. Keywords— DNS over HTTPS, Machine Learning, Detecção de Intrusão, Ensemble Stacking, Explicabilidade (XAI).

1. Introdução e contextualização

A adoção do protocolo DNS over HTTPS (DoH) trouxe avanços significativos para a privacidade dos usuários ao criptografar as requisições de resolução de nomes, ocultando-as em meio ao tráfego HTTPS padrão. Contudo, essa mesma característica introduziu um desafio crítico para a detecção de intrusões, uma vez que malwares e ferramentas de exfiltração de dados passaram a utilizar túneis DoH para contornar inspeções de segurança corporativas.

Para mitigar essa ameaça, o artigo de referência [Inserir Nome dos Autores, Ex: Zebin et al., 2022] propõe um sistema robusto de detecção de intrusão baseado em Machine Learning. A solução dos autores foca em analisar características estatísticas do fluxo de rede (como tamanho e duração dos pacotes) sem a necessidade de descriptografar o tráfego. O núcleo metodológico do estudo é uma arquitetura de classificação empilhada (Stacked Ensemble). Para lidar com o massivo desbalanceamento de dados entre o tráfego comum e as requisições DoH, o estudo original divide a classe majoritária em blocos paralelos e aplica a sobreamostragem sintética (SMOTE). A primeira camada do modelo treina três sub-modelos de Random Forest, cujas saídas alimentam um meta-classificador de Regressão Logística. Aliando essa estrutura a ferramentas de Inteligência Artificial Explicável (XAI) baseadas em valores SHAP, os autores relatam ter alcançado uma acurácia excepcional de 99,9.

O presente relatório detalha a replicação metodológica e a análise crítica de um sistema de detecção de intrusão baseado em Machine Learning, proposto na literatura de referência. O objetivo central desta fase inicial do projeto foi reconstruir fielmente a arquitetura de classificação empilhada (Stacking) e a camada de explicabilidade (XAI) do artigo original, avaliando sua real eficácia e reprodutibilidade. Além de buscar a validação dos 99,9% de acurácia reportados pelos autores, este trabalho estabelece a base para uma proposta de melhoria contínua baseada em generalização e redução de dimensionalidade, que será aplicada a um novo conjunto de dados.

2. Metodologia de Replicação

Para garantir o rigor científico e a fidelidade ao estudo de referência, o código foi reconstruído a partir da correção de lacunas no repositório original dos autores, implementando as seguintes etapas no pipeline de dados e modelagem:

- Pré-processamento Cirúrgico e Montagem do Dataset:** O conjunto de dados foi construído a partir da união estratégica de três arquivos brutos (l1-nondoh.csv, l2-benign.csv e l2-malicious.csv). O arquivo genérico l1-doh.csv foi descartado, pois seus dados já se encontravam devidamente segregados nas classes da camada 2. A coluna alvo (Label) foi rigorosamente mantida como gabarito da classificação. Em seguida, removemos metadados de rede (como IPs e portas) que poderiam enviesar o modelo, isolando estritamente as 29 características estatísticas de comportamento validadas pelo artigo. Por fim, aplicou-se a normalização Min-Max.
- Engenharia de Memória e Particionamento:** Devido ao volume massivo da classe majoritária (Non-DoH, com quase 900 mil instâncias), implementou-se uma divisão estrutural. A classe foi particionada em 3 blocos paralelos, garantindo a viabilidade computacional do treinamento sem perda de representatividade.
- Tratamento de Desbalanceamento (SMOTE):** A classe minoritária (Benign-DoH) sofreu sobreamostragem através da técnica SMOTE. É imperativo destacar que este procedimento foi aplicado exclusivamente aos dados de treinamento, preservando a integridade do conjunto de teste (10%) e evitando qualquer forma de vazamento de dados (data leakage).
- Validação Cruzada (10-Fold CV):** Para treinar o meta-classificador de forma isenta, empregou-se uma Validação Cruzada de 10 dobras. Essa abordagem gerou as probabilidades Out-of-Fold (OOF), garantindo que o algoritmo de segundo nível aprendesse a partir de predições feitas sobre dados invisíveis aos sub-modelos.
- Arquitetura Stacking Oficial:** A camada base foi constituída por 3 sub-modelos Random Forest (configurados com 10 estimadores e profundidade máxima de 5), treinados independentemente nos blocos particionados. As saídas probabilísticas alimentaram o Meta-Classificador baseado em Regressão Logística.
- Explicabilidade (XAI):** O modelo final foi integrado à biblioteca SHAP (via ExplainerDashboard) para permitir a interpretação global e local das predições, assegurando a transparência do processo decisório.

3. Discussão Crítica dos Resultados

Os resultados obtidos na replicação demonstraram o alto poder preditivo da arquitetura base, mas também revelaram limitações práticas importantes. O modelo atingiu uma acurácia geral excelente de 98%, com métricas de precisão e recall beirando a perfeição (98% e 99%, respectivamente) para a detecção de ataques maliciosos e tráfego HTTPS padrão (Non-DoH).

Métrica	Score
accuracy	0.989
precision	0.984
recall	0.964
f1	0.974
roc_auc_score	0.998
pr_auc_score	0.996
log_loss	0.103

Tabela 1: Métricas de avaliação de desempenho do modelo Stacking Ensemble. O sistema atingiu uma acurácia global de 98,9% e altos índices de precisão e recall gerais, comprovando a eficácia preditiva da arquitetura

- O Problema do Desbalanceamento Extremo:** Apesar da alta eficácia geral, a replicação evidenciou uma vulnerabilidade na classe minoritária. O recall para a classe Benign-DoH atingiu apenas 45%. Para investigar esse comportamento, conduzimos um experimento inserindo o parâmetro `classweight='balanced'` no Meta-Classificador (Regressão Logística). O ajuste elevou o recall do tráfego benigno para 92%; contudo, a precisão despencou para 22%, gerando aproximadamente 6.000 falsos positivos (instâncias normais classificadas erroneamente como benignas).

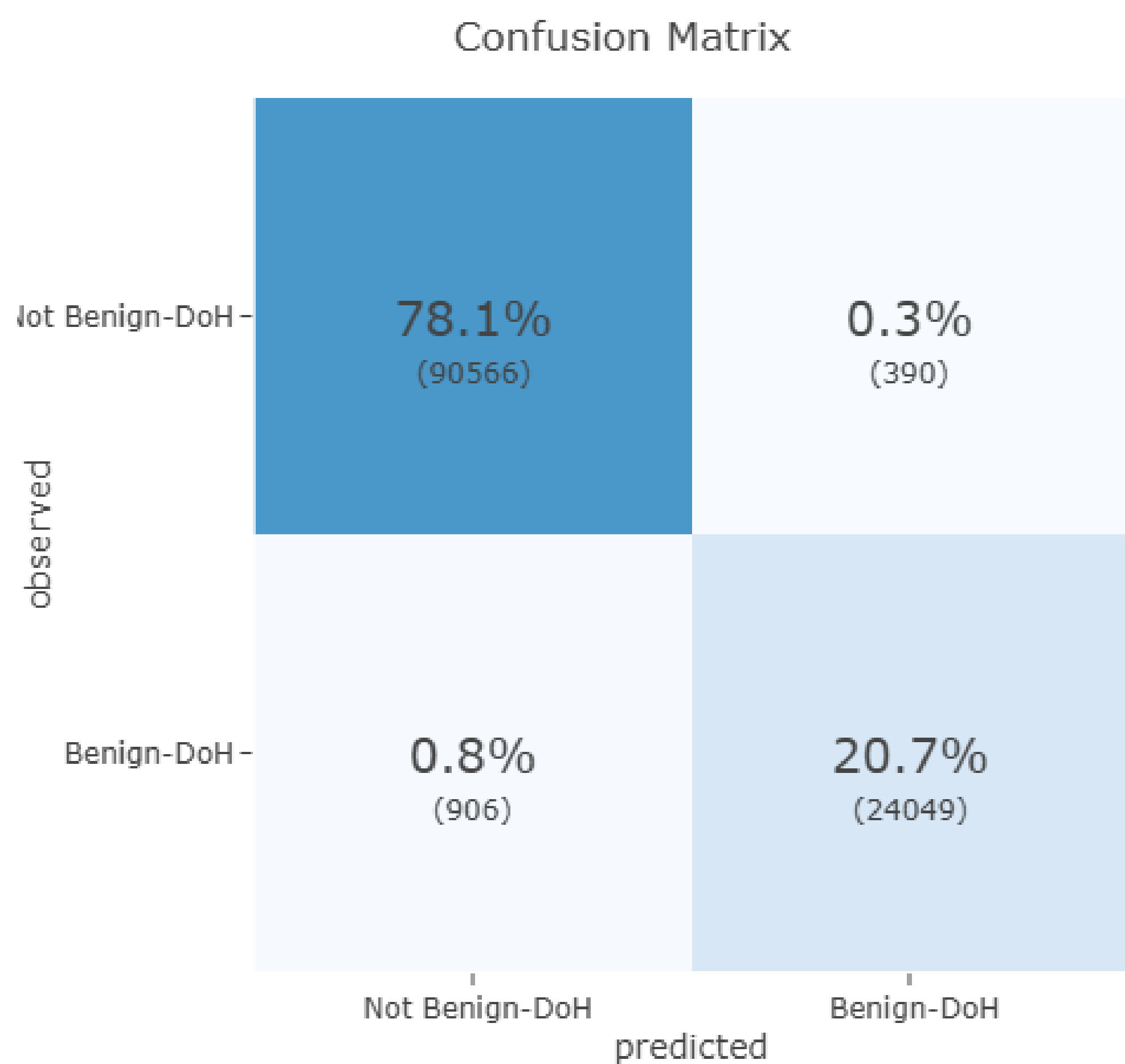


Figura 1: Visão binária da matriz de confusão gerada pelo painel XAI. A imagem evidencia o desafio de detecção da classe minoritária (Benign-DoH), ilustrando o impacto do desbalanceamento extremo do conjunto de dados original.

- A Crise de Reprodutibilidade na IA:** Diante do trade-off severo, optou-se por manter a arquitetura purista original, aceitando o modelo de 98%. O desvio em relação à acurácia de 99,9% relatada no artigo de referência é um reflexo direto do que a comunidade acadêmica denomina de "Crise de Reprodutibilidade". A extrema dificuldade de generalizar a detecção da classe minoritária, mesmo com SMOTE, expõe o desbalanceamento massivo do dataset (uma proporção de quase 45:1). A diferença métrica sugere que os autores originais podem ter se beneficiado de sementes aleatórias altamente favoráveis ou otimizado hiperparâmetros de forma implícita (não documentada). Dessa forma, nossos resultados de laboratório não representam uma falha de código, mas sim uma validação honesta, crítica e

realista das limitações da metodologia sob o escrutínio de boas práticas de separação de dados.

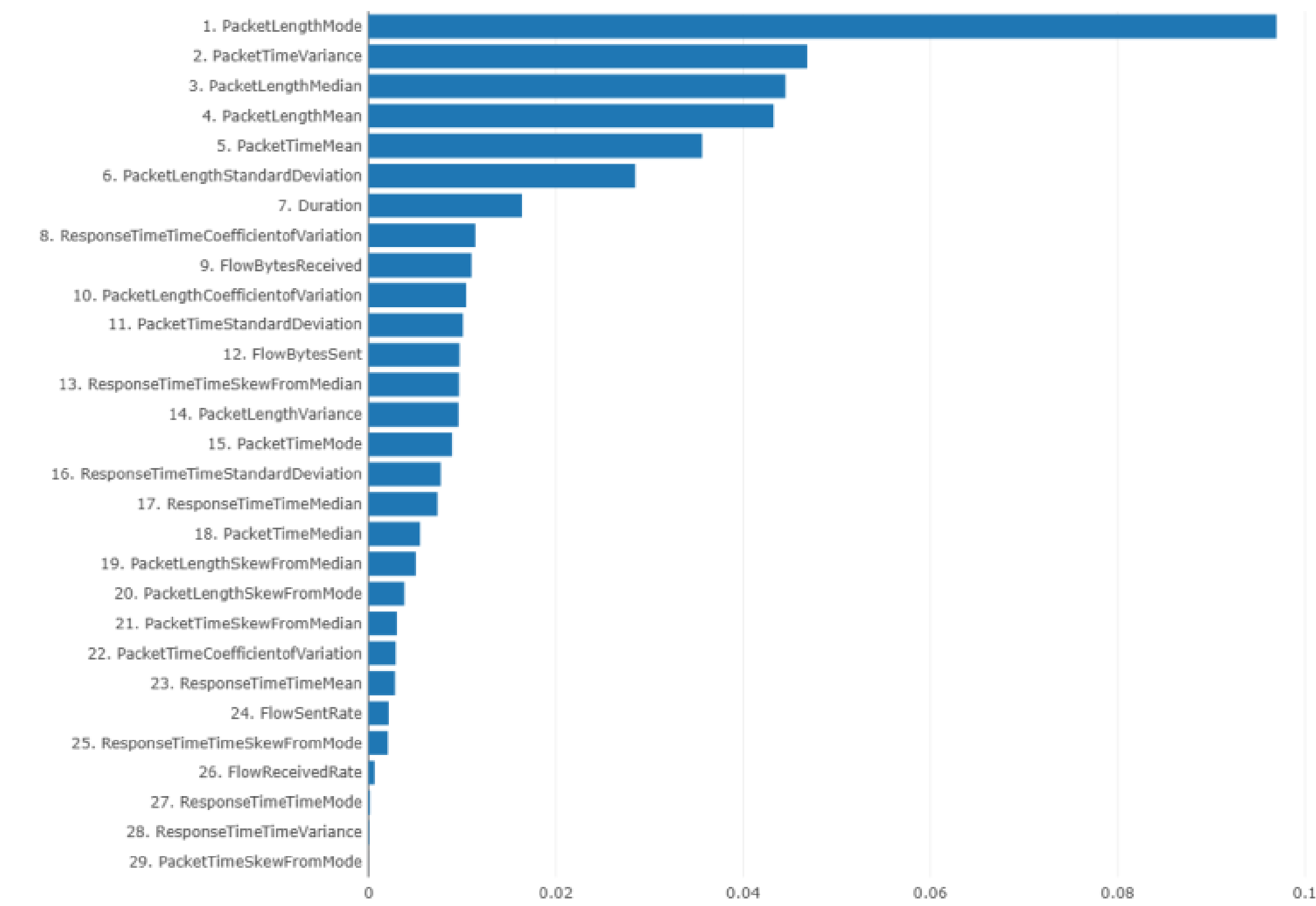


Figura 2: Importância das características (Feature Importance) baseada nos valores médios absolutos de SHAP. Atributos como o modo do comprimento do pacote (PacketLengthMode) e a variância do tempo do pacote (PacketTimeVariance) demonstram maior impacto nas decisões do modelo.

4. Fase 2: Proposta de melhoria e validação em novo dataset

Para replicar os experimentos originais em um novo conjunto de dados, decidimos utilizar a junção do CIRA-CIC-DoHBrw-2020, que foi originalmente utilizado no artigo-alvo deste trabalho, com o DoH-Tunnel-Traffic-HKD: o CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD.

O conjunto de dados combinado foi escolhido em vez do DoH-Tunnel-Traffic-HKD porque a arquitetura proposta originalmente no artigo faz mais sentido para datasets desbalanceados, o que não é o caso do HKD. Já que o CIRA-CIC-DoHBrw-2020 é o dataset mais utilizado para aplicações de detecção de ataques do tipo DoH e possui uma quantidade significativamente maior de fluxos de rede do tipo Non-DoH em comparação com outras classes, sua versão unificada com o DoH-Tunnel-Traffic-HKD nos permitiu testar a técnica proposta em um contexto de desbalanceamento de dados, mas dessa vez com instâncias maliciosas adicionais geradas por 3 novos tipos de ferramentas de tunneling: `dnstt`, `tcp-over-dns`, e `tuns`. Isso nos permitiu testar a consistência tanto nas métricas de performance em comparação com o trabalho original, quanto na explicabilidade das decisões, além de aumentar o alcance da metodologia a outras ferramentas.

O dataset unificado conta com três camadas com diferentes atributos preditores, uma a mais em relação ao CIRA-CIC-DoHBrw-2020. A primeira camada é voltada para a classificação binária de tráfego DoH ou Non-DoH; a segunda utiliza apenas as instâncias DoH da primeira camada, detalhando sua classificação entre Benign-DoH e Malicious-DoH; já a terceira e última camada utiliza apenas as instâncias classificadas como Malicious-DoH da segunda camada, de forma que as possíveis classes sejam as ferramentas de tunneling que geraram o respectivo ataque malicioso: `dns2tcp`, `dnscat2`, `iodine`, `dnstt`, `tcp-over-dns`, e `tuns`.

Com o objetivo de replicar os experimentos originais no novo conjunto de dados, unimos as instâncias Non-DoH da primeira camada com a segunda camada (que possui as classes Benign-DoH e Malicious-DoH), de forma que pudéssemos classificar os fluxos de rede entre essas 3 classes. Expandindo os experimentos originais, unificamos também as instâncias Non-DoH da primeira camada com as instâncias Benign-DoH da segunda camada e com a terceira camada completa, o que nos gerou um problema de classificação com 8 classes. Embora a quantidade de classes seja diferente, a quantidade de instâncias é a mesma.

- Pré-processamento e balanceamento:** O novo dataset muda a proporção de balanceamento original por acrescentar apenas instâncias maliciosas aos dados. Para replicar o experimento original, repetimos as mesmas estratégias para o balanceamento, como dividir os dados em conjuntos de treino (90%) e teste (10%). Foi realizada validação cruzada com 10 dobras. Para cada dobra, as instâncias majoritárias (Non-DoH) dos dados de treino foram divididas em partições como estratégia de balanceamento, e para cada partição, as classes minoritárias foram repetidas e aumentadas (oversampling) usando o SMOTE. Em seguida, normalizamos usando o MinMax Scaler.
- Layer 1 + Layer 2 (3 classes, replicação da metodologia):** As proporções entre as classes para o CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD antes do balanceamento são 45:18:1, em contraste aos 45:12:1 do CIRA-CIC-DoHBrw-2020. Os autores dividem a classe majoritária em 3 partições, assim como nós. Em nosso experimento, essa divisão nos levou à proporção 15:18:1 para cada subconjunto antes do oversampling com SMOTE. Após o balanceamento, chegamos a 1:1:1, em contraste aos 15:12:12 obtido pelos autores.
- Layer 1 + Layer 2 + Layer 3 (8 classes, alteração na arquitetura):** Os dados de treino do segundo experimento, por outro lado, seguiram uma proporção mais complexa de se trabalhar com: 45:8:2:2:2:2:1:1. Isso porque dividir as instâncias majoritárias (Non-DoH) em apenas 3 conjuntos ainda resultaria em um desbalanceamento considerável e precisaríamos gerar muitas instâncias sintéticas para muitas classes. Nossa estratégia foi, portanto, dividir as instâncias Non-DoH em 5 partições em vez de 3, o que nos levou à proporção 9:8:2:2:2:2:1:1 para cada subconjunto. Após a aplicação do SMOTE, chegamos à proporção de 1:1:1:1:1:1:1:1.
- Modelos de Classificação:** De forma semelhante ao artigo base, cada um dos subconjuntos balanceados foi alimentado a um modelo de **Random Forest**. No primeiro experimento, em que existem 3 partições balanceadas para cada dobra, utilizou-se 3 modelos para a classificação de fluxo Non-DoH, Benign-DoH, e Malicious-DoH, um para cada subconjunto. Já no segundo, em que existem 5 partições balanceadas para cada dobra, utilizou-se 5 modelos para a classificação de fluxo Non-DoH, Benign-DoH, ou alguma das 6 ferramentas de tunneling mencionadas (labels MaliciousDoH-dns2tcp, MaliciousDoH-iodine, MaliciousDoH-dnstt, MaliciousDoH-dnscat2, MaliciousDoH-tcp-over-dns, MaliciousDoH-tuns).
- Meta-Modelo:** As probabilidades geradas por cada Random Forest alimentam um meta-modelo, que é uma Regressão Logística. Esse passo a passo segue a mesma estratégia do trabalho original, diferindo apenas no segundo experimento, já que o meta modelo recebe as probabilidades de 5 classificadores (já que existem 5 subconjuntos de dados) em vez de 3.
- Avaliação de Desempenho** Analisamos o desempenho na predição por classe considerando os dois experimentos.
 - Classificação 3 classes:** Alcançamos uma acurácia geral de 0.99 e as métricas por classe conforme descritas na Tabela 2.

Classe	Precision	Recall	F1
Benign-DoH	0.76	0.93	0.83
Malicious-DoH	1.0	1.0	1.0
Non-DoH	1.0	0.99	1.0

Tabela 2: Métricas de avaliação de desempenho do modelo Stacking Ensemble para o primeiro experimento com o dataset CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD (3 classes)

Podemos perceber que a a classe minoritária (Benign-DoH) obteve resultados pouco satisfatórios para a precisão. O resultado para a classe MaliciousDoH, embora tenha sido arredondado pelo scikit-learn para 1.0 nas métricas apresentadas, atingiu um resultado curioso: apenas 51 instâncias maliciosas foram incorretamente classificadas em contraste às 35.471.

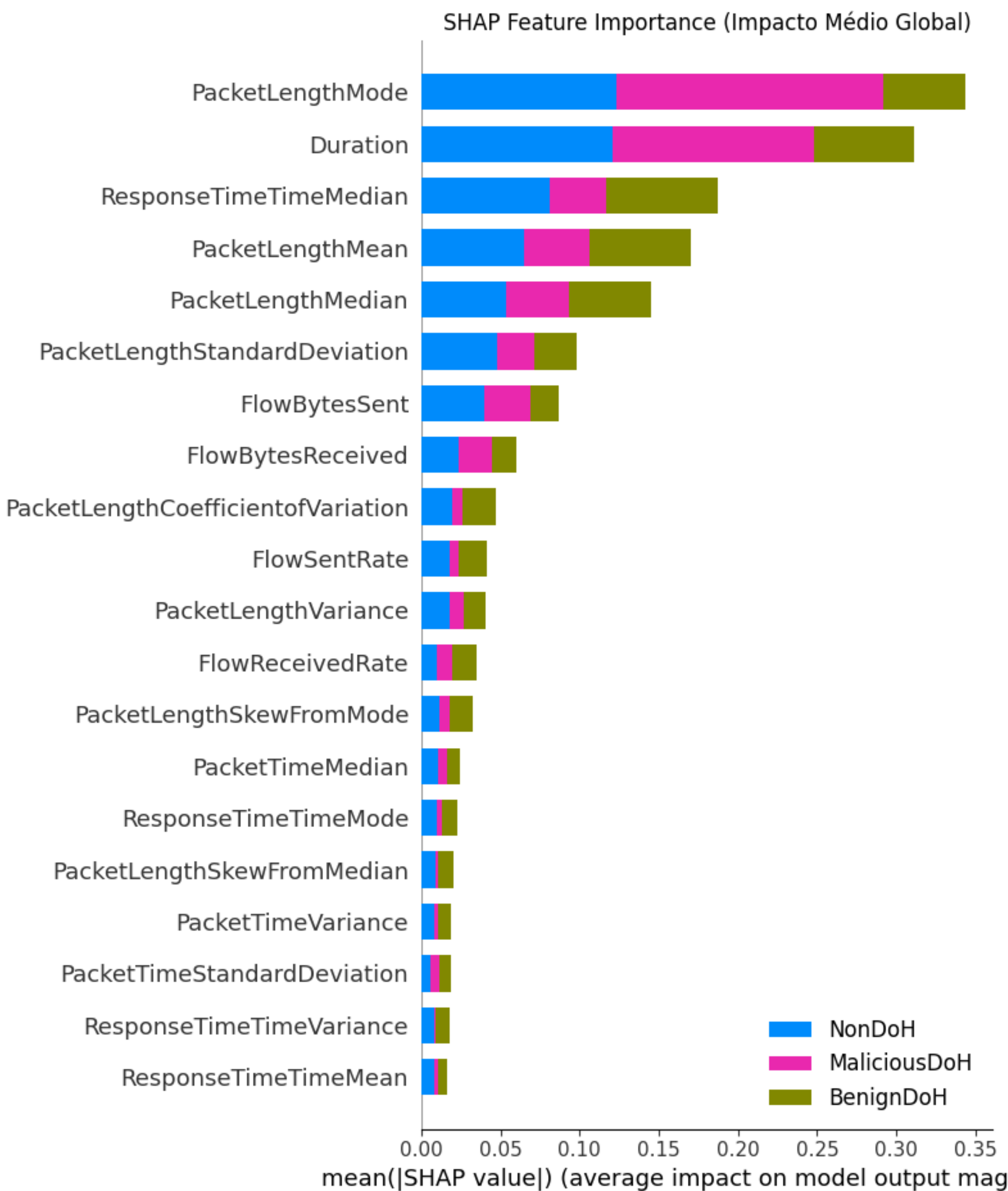


Figura 3: Feature Importances para o experimento de classificação com 3 classes

Em relação à explicabilidade, podemos perceber a partir do plot de feature importances gerado através da biblioteca SHAP, que as features "Duration" e "PacketLength-Mode" permanecem como as mais importantes para a tomada de decisão do modelo para o novo dataset (em comparação com o CIRA-CIC-DoHBrw-2020), apenas em posições trocadas no ranking de importância.

- Classificação 8 classes:** Também alcançamos uma acurácia geral de 0.99 para o segundo experimento, e obtivemos as métricas por classe descritas na Tabela 3.

Classe	Precision	Recall	F1
Benign-DoH	0.84	0.92	0.87
Malicious-DoH-dns2tcp	1.0	0.98	0.99
Malicious-DoH-dnscat2	0.87	0.93	0.90
Malicious-DoH-dnstt	1.0	1.0	1.0
Malicious-DoH-iodine	0.92	0.94	0.93
Malicious-DoH-tcp-over-dns	1.0	1.0	1.0
Malicious-DoH-tuns	1.0	1.0	1.0
Non-DoH	1.0	1.0	1.0

Tabela 3: Métricas de avaliação de desempenho do modelo Stacking Ensemble para o segundo experimento com o dataset CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD (8 classes)

Assim como no experimento anterior, a classe Benign-DoH, que é a minoritária, foi a que obteve os piores resultados. Uma observação interessante é que os resultados

obtidos para os ataques gerados com iodine e dnscat2, que os autores do artigo original identificam como tendo comportamentos parecidos na análise de explicabilidade, foram os ataques que segundo a nossa análise, obtiveram os piores resultados. Isso ajuda a indicar que de fato, o comportamento das duas ferramentas de tunneling pode ser parecido. Já em relação ao plot de feature importances, conforme observa-se na Figura 4, "Duration" que era um dos atributos mais importantes para a tomada de decisão dos modelos nos experimentos anteriores foi considerado um dos menos importantes. Por outro lado, "PacketLengthMode" permanece no topo de importância. Isso pode indicar que embora a duração do fluxo de rede ajude a identificar se uma instância é um ataque ou não, ela não é crucial na identificação da ferramenta que gerou o ataque (para afirmar isso, mais experimentos com outros conjuntos de dados são necessários).

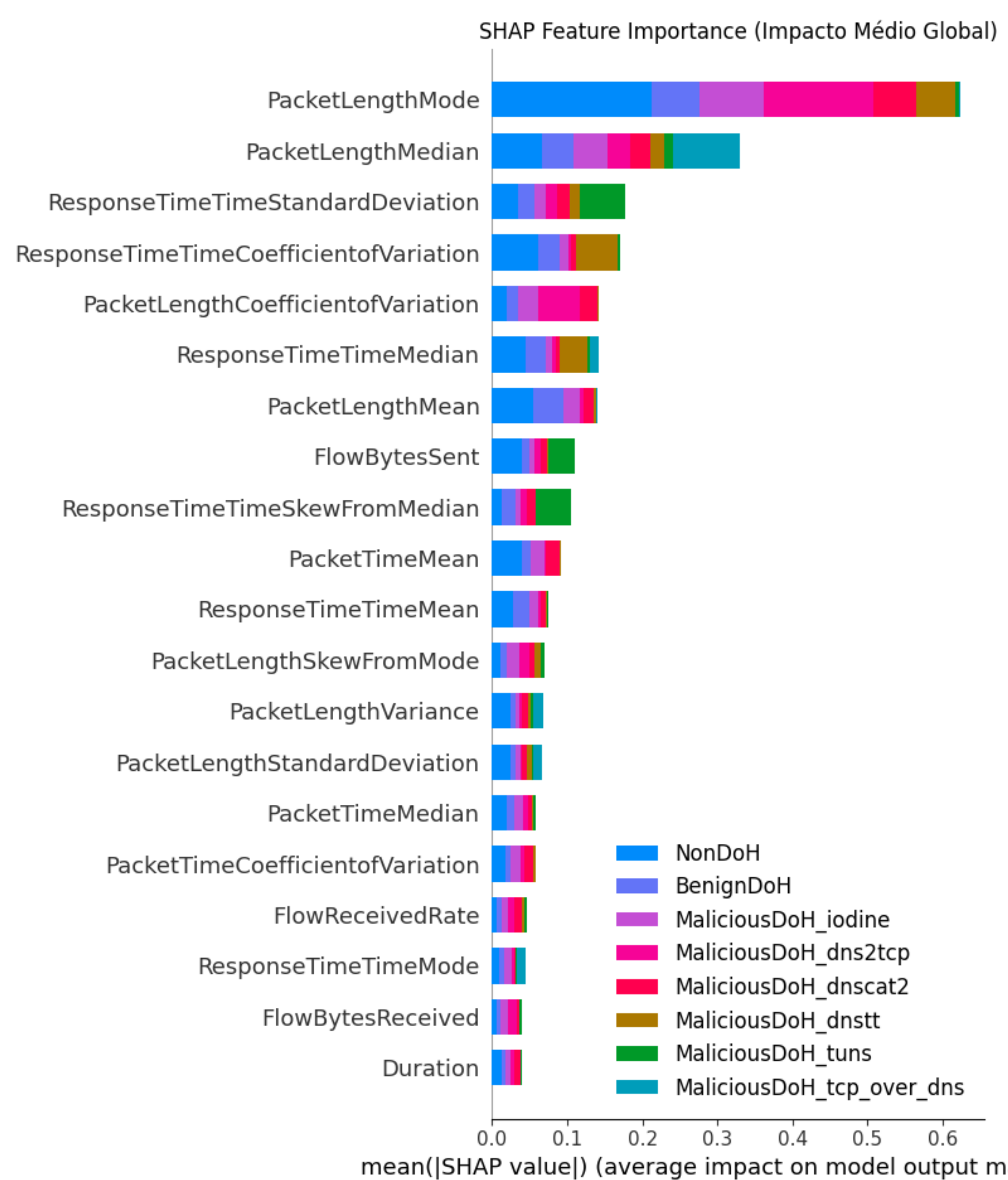


Figura 4: Feature Importances para o experimento de classificação com 8 classes

5. Experimento adicional: Testando somente um Random Forest

Nesta etapa, foi desenvolvido um modelo de classificação utilizando o algoritmo Random Forest como o intuito de experimentar e observar os valores resultantes, sem o uso de técnicas de ensemble em múltiplos níveis.

- **Pré-processamento dos Dados** :Os dados utilizados foram provenientes de múltiplas fontes, incluindo os conjuntos L1, L2, L3 e um dataset adicional contendo tráfego DNS tunelado. Inicialmente, foi realizada a filtragem e balanceamento de classes específicas, seguido da unificação dos datasets com base nas colunas comuns.
- Em seguida, foi realizada a transformação do problema em uma tarefa de classificação multiclasse, onde os rótulos foram mapeados para três categorias principais: benigno (0) , não-DoH (1) e malicioso (2).
- **Divisão dos Dados** O conjunto de dados foi dividido em três subconjuntos:
 - Treinamento (70%)
 - Validação (15%)
 - Teste (15%)A divisão foi realizada de forma estratificada, preservando a proporção das classes em cada subconjunto.
- **Treinamento do Modelo**

O modelo Random Forest foi treinado utilizando todas as features numéricas disponíveis, após normalização com o StandardScaler. Posteriormente, foi realizada uma análise de importância das variáveis, permitindo a seleção das 10 features mais relevantes.

- **Avaliação de Desempenho**

A avaliação foi realizada utilizando métricas de classificação como precisão, recall e F1-score, além da matriz de confusão para análise detalhada dos erros. Os resultados indicaram que o uso das features mais importantes mantém desempenho competitivo, enquanto a remoção dessas features impacta negativamente a performance do modelo, evidenciando sua relevância.

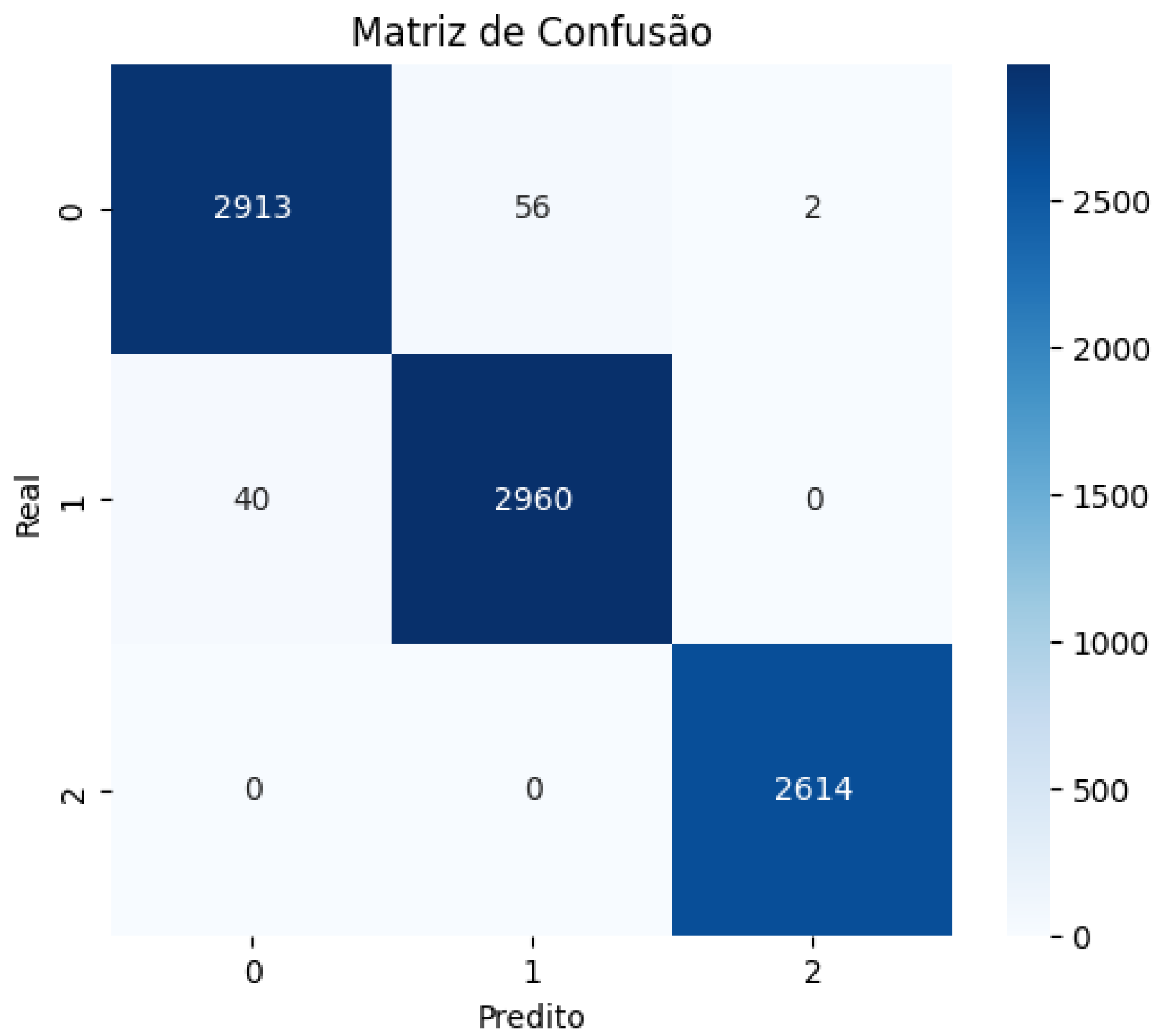


Figura 5: Matriz de confusão do modelo Random Forest

- **Análise de Overfitting** Para verificar a robustez do modelo, foram realizadas a comparação entre a acurácia no conjunto de treino e teste, além de treinamento com rótulos embaralhados(shuffle test) No teste de embaralhamento, o modelo apresentou desempenho próximo ao aleatório, indicando ausência de vazamento de dados e boa generalização.
- **Análise de Features Individuais** Analisar features individualmente permitiu avaliar o poder discriminativo individual de determinadas variáveis.

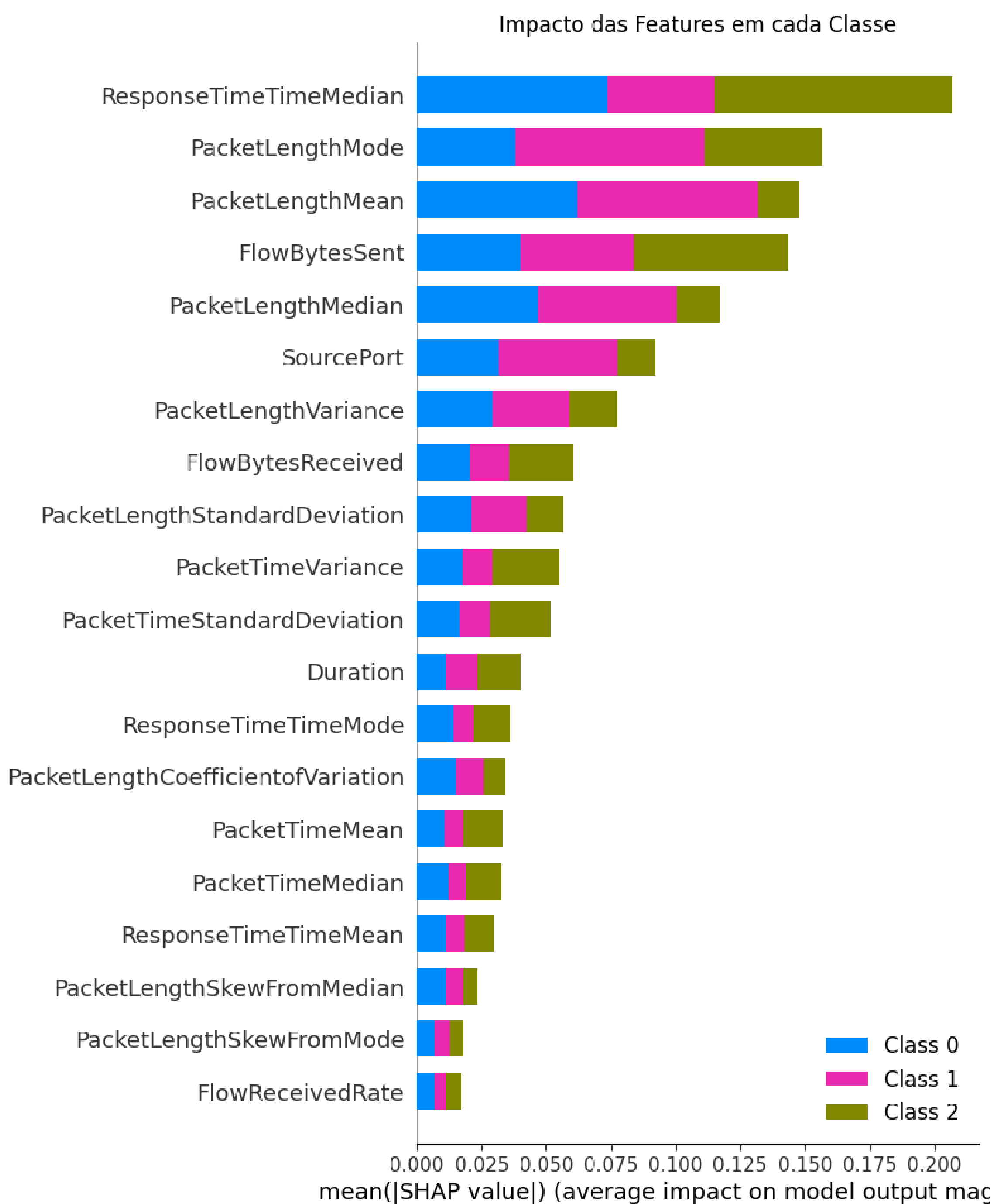


Figura 6: Gráfico de barras com os valores médios absolutos de SHAP, demonstrando o impacto e a contribuição de cada variável na classificação para as Classes.

Diferentemente do treinamento com o metamodelo, observou-se que, ao utilizar o algoritmo Random Forest de forma isolada, a variável de maior importância foi o tempo médio de resposta.

Por fim, verifica-se que o modelo proposto, mesmo empregando apenas o algoritmo Random Forest como abordagem inicial, apresentou um desempenho semelhante, porém inferior ao reportado na literatura base. Esses resultados indicam que, embora o modelo seja competitivo, a ausência de técnicas mais sofisticadas, como estratégias de combinação de modelos, pode limitar seu desempenho em relação a abordagens mais avançadas.

Referências

[1] A. Author et al., "An Explainable AI-Based Intrusion Detection System for DNS Over HTTPS (DoH) Attacks,"IEEE Access, 2023.

[2] H. S. Anderson et al., "CIRA-CIC-DoHBrw-2020 Dataset,"Canadian Institute for Cybersecurity (CIC), University of New Brunswick, 2020.

[3] DoH-Tunnel-Traffic-HKD Dataset, "DNS over HTTPS Tunnel Traffic Dataset,"GitHub Repository. Available: <https://github.com/doh-traffic-dataset/DoH-Tunnel-Traffic-HKD/>

[4] L. Breiman, "Random Forests,"Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[5] S. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions,"in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017.

[6] Scikit-learn Developers, "Scikit-learn: Machine Learning in Python,"Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] J. Shlens, "A Tutorial on Principal Component Analysis,"arXiv preprint arXiv:1404.1100, 2014.